Stage 5 实验报告

于越洋

1 实验内容

Step 10

词法语法分析

在 `frontend/ast/tree.py` 中，我修改了现有的 AST 节点类型：

- `Program`： 添加了 `global_decls` 和 `funcs_and_decls` 两个成员函数，分别以列表形式返回所有全局变量声明、所有函数和全局变量声明。

我在 `frontend/parser/ply_parser.py` 中定义了新的规则：

- `p_global_decl`： 表示全局变量的声明。
- `p_global_decl_init`： 表示带有初始化语句的全局变量的声明。

并修改了现有的规则：

- `p_program`： 现在一个程序可以由多个函数和全局变量声明组成。

语义分析

在 `frontend/typecheck/namer.py` 中，我修改了 `Namer` 类中的以下成员函数：

- `visitProgram`： 现在将先对所有的全局变量声明进行语义分析，将其加入全局作用域，然后再对所有函数进行语义分析。

中间代码生成

在 `utils/tac/tacinstr.py` 中，我添加了以下类型：

- `LoadSymbol`： 表示加载全局变量地址的指令，包含全局变量符号 `symbol` 和目标临时变量 `dst`，TAC 语句设定为 `dst = LOAD_SYMBOL symbol`。
- `LoadData`： 表示从内存中加载数据的指令，包含基址 `addr`、偏移 `offset`、目标临时变量 `dst`，TAC 语句设定为 `dst = LOAD addr, offset`。
- `StoreData`： 表示向内存中存储数据的指令，包含基址 `addr`、偏移 `offset`、源临时变量 `src`，TAC 语句设定为 `STORE src, addr, offset`。

在 `utils/tac/tacprog.py` 中，我在 `TACProg` 类中添加了全局变量列表 `self.globalDecls`，并在 `printTo` 函数中添加了对全局变量的打印。

在 `utils/tac` 中，我创建了 `tacglobaldecl.py` 文件，包含 `TACGlobalDecl` 类，与 `tacfun.py` 中的 `TACFunc` 类类似，用于全局变量的表示和打印，方便后续处理。

在 `frontend/symbol/varsymbol.py` 中，我为 `VarSymbol` 类添加了以下成员函数

- `getInitValue`： 用于获得变量的初始化值。
- `isGlobalVar`： 用于判断是否为全局变量。
- `isInitialized`： 用于判断是否进行了初始化。

在 `frontend/tacgen/tacgen.py` 中，我为 `TACFuncEmitter` 类添加了以下成员函数：

- `emitLoadSymbol`： 用于生成加载全局变量地址的语句。
- `emitLoadData`： 用于生成从内存中加载数据的语句。
- `emitStoreData`： 用于生成向内存中存储数据的语句。

我还修改了 `TACGen` 类的以下成员函数：

- `visitIdentifier`： 增加了对全局变量的处理，若遇到全局变量则先加载其地址，再从内存中加载其值。
- `visitAssignment`： 增加了对全局变量的处理，若赋值目标为全局变量，则先加载其地址，再向内存中写入被赋的值。
- `transform`： 增加了对于全局变量的处理，处理完全部全局变量后再处理函数。

目标代码生成

我在 `utils/riscv.py` 中为 `Riscv` 类添加了以下成员类：

- `LoadSymbol`： 表示加载全局变量地址的指令，将全局变量符号对应的地址加载进目标寄存器，使用 `la` 指令实现。
- `LoadData`： 表示从内存中加载数据的指令，将数据从内存中指定地址处加载到目标寄存器，使用 `lw` 指令实现。
- `StoreData`： 表示向内存中存储数据的指令，将数据从源寄存器写入内存中指定地址处，使用 `sw` 指令实现。

我还修改了 `backend/riscv/riscvasmemitter.py` 中的 `RiscvAsmEmitter` 类的构造函数，添加了参数 `globalDecls` 表示全局变量声明列表，并添加了打印数据段的部分，将初始化赋值非 0 的全局变量存入 `.data` 段，其余存入 `.bss` 段。

Step 11

词法语法分析

在 `frontend/ast/tree.py` 中，我添加了新的 AST 节点类型：

- `IndexExpression`： 用于表示索引表达式，包含数组基址表达式 `base`，索引 `index`。
- `TArray`： 表示数组节点类型，包含基类型 `base` 和长度 `size`。

我还为 `Declaration` 节点添加了 `__str__` 函数，便于在语法树中打印声明节点。

我在 `frontend/lexer/lex.py` 中添加了 `t_LBracket='['` 和 `t_RBracket=']'` 用来表示数组索引符号。

我也在 `frontend/parser/ply_parser.py` 修改了变量声明相关的规则，原有的 `p_declaration_init`、`p_declaration`、`p_global_decl`、`p_global_decl_init` 改为如下形式

- `p_variable_declaration`： 表示变量声明。
- `p_variable_declaration_init`： 表示带有初始化语句的变量声明。
- `p_array_declaration`： 表示数组声明，递归处理，记录数组的每一纬度信息。
- `p_array_declaration_init`： 表示可能带有初始化语句的数组声明，根据递归处理声明的结果，逐级创建数组的类型表示，如 `int a[3][4][5]` 的类型将被创建为 `TArray(TArray(TArray(TInt, 5), 4), 3)`，然后用该类型信息创建声明 `Declaration` 节点。
- `p_global_decl`： 表示全局变量/数组的声明，直接导向 `p_declaration`。
- `p_declaration`： 表示变量/数组声明，导向 `p_variable_declaration` 或 `p_array_declaration_init`。

我还修改了 `p_expression_precedence` 规则，添加了对于数组使用的支持，即递归建立索引表达式节点 `IndexExpression`。

语义分析

在 `frontend/typecheck/namer.py` 中，我在 `Namer` 类中添加了以下成员函数：

- `visitIndexExpression`： 用于完成对索引表达式的语义分析，先对每一纬度的索引进行语义分析，再对数组符号进行语义分析，并在索引表达式的维度与数组维度不匹配时报错。

我还修改了 `visitDeclaration` 函数，添加了对于数组声明的每一维度长度信息的检查，若有某一纬度长度不是正数则报错。

中间代码生成

我修改了 `utils/tac/tacglobaldecl.py` 中的 `TACGlobalDecl` 类，为其添加了表示变量类型的成员变量 `self.type`。

我在 `utils/tac/tacinstr.py` 中添加了 `Alloc` 类型，表示内存分配语句，用于为数组分配内存空间，包含目标临时变量 `dst` 和空间大小 `size`，TAC 语句设定为 `dst = ALLOC size`。

在 `frontend/tacgen/tacgen.py` 中，我为 `TACFuncEmitter` 类添加了成员函数 `emitAlloc`，生成一条 `Alloc` 语句。

我还修改了 `TACGen` 类中的以下成员函数：

- `visitDeclaration`： 添加了对于数组类型的处理，即若声明的类型为数组，则分配内存空间。
- `visitAssignment`： 添加了对于索引表达式的处理，即若赋值目标是数组元素，则先计算目标的内存地址，再生成内存写入语句。

并在 `TACGen` 类中添加了成员函数 `visitIndexExpression`，用于处理对数组元素的访问，将先计算数组元素的内存地址，再生成加载内存内容的语句。

目标代码生成

我在 `utils/riscv.py` 中为 `Riscv` 类添加了以下成员类：

- `NativeAddi`： 实现 `addi` 指令，便于对栈指针的操作。
- `Alloc`： 内存分配指令，存储分配的内存大小 `size`。

我在 `backend/riscv/riscvasmemitter.py` 中的 `RiscvInstrSelector` 类中添加了成员函数 `visitAlloc`，在 Risc-V 代码序列中添加内存分配语句。

我还修改了 `backend/riscv/riscvasmemitter.py` 中 `RiscvSubroutineEmitter` 类中的 `emitAsm` 函数，对内存分配语句进行判断，若遇到则使用 `addi` 指令修改栈指针。

Step 12

词法语法分析

我修改了 `frontend/ast/tree.py` 中 `Declaration` 节点的定义，将成员变量 `init_expr` 改为 `init_exprs`，便于处理数组初始化时的多个初始化值。

我在 `frontend/parser/ply_parser.py` 添加和修改了数组初始化和传参相关的规则：

- `p_array_init`： 递归处理数组初始化的数值列表，记录所有初始化值。
- `p_array_init_empty`： 处理数组的空初始化列表。
- `p_array_parameter`： 处理数组参数，递归获取每维度信息。
- `p_first_size`： 用于获得数组参数的第一维长度。
- `p_first_size_empty`： 当省略了数组参数的第一维长度时，默认设置为 0。

- `p_parameter`：用于处理函数参数，对于普通变量参数，直接处理，建立 `Parameter` 节点；对于数组参数，递归创建 `TArray` 类型，最后建立 `Parameter` 节点。

此外，我也修改了所有和 `Declaration` 节点相关的规则，以适应节点的最新定义。

语义分析

在 `frontend/typecheck/namer.py` 中，我在 `Namer` 类中修改了 `tranform` 和 `visitDeclaration` 成员函数，以支持最新的 `Declaration` 节点定义，此外还补全了处理函数参数和索引表达式时对 `Identifier` 节点对应符号的设置。

我补全了 `frontend/typecheck/typer.py` 文件中的 `Typer` 类，在其中完成了所有的类型检查工作，实现方式与 `namer.py` 类似，类型检查未通过时将会报错。

中间代码生成

我修改了 `utils/tac/tacglobaldecl.py` 中的 `TACGlobalDecl` 类，将成员变量 `self.init_val` 改为 `self.init_vals`，支持数组的初始化。

在 `frontend/tacgen/tacgen.py` 中，我为 `TACGen` 类添加了成员变量 `self.addFillNFunction`，表示是否要添加 `fill_n` 函数，该函数用于完成一块内存空间的初始化，如果源代码涉及到数组的初始化，则 `self.addFillNFunction` 将被设置为 `True`，成员函数 `transform` 中将会添加对 `fill_n` 函数的实现。

我还修改了 `TACGen` 类中的 `visitIdentifier` 成员函数，将数组和普通变量的标识符访问分开处理；以及修改了 `visitDeclaration` 成员函数，添加了对于数组初始化的处理。

目标代码生成

我修改了在 `backend/reg/bruteregalloc.py` 中的 `BruteRegAlloc` 类的成员函数 `localAlloc`，在函数调用前，将函数调用完成后还需要使用的寄存器值存入栈中，以免函数调用修改数组首指针寄存器导致函数调用完成后数组访问错误。

我修改了 `backend/riscv/riscvasmemitter.py` 中 `RsicvAsmEmitter` 类的构造函数，修改了将变量/数组放入 `.data` 段和 `.bss` 段的条件，以支持数组的初始化情况。

2 思考题

Step 10

Question 1

> 写出 `la v0, a` 这一 RiscV 伪指令可能会被转换成哪些 RiscV 指令的组合（说出两种可能即可）。

Answer：可能性 1：绝对寻址

```
auipc v0, %hi(a)        # 加载地址的高 20 位
addi v0, v0, %lo(a)     # 加载地址的低 12 位
```

可能性 2：PC 相对寻址

```
auipc  v0, %pcrel_hi(a)     # 加载相对于 PC 的高 20 位偏移量
addi   v0, v0, %pcrel_lo(a) # 加载相对于 PC 的低 12 位偏移量
```

Step 11

Question 1

> C 语言规范规定，允许局部变量是可变长度的数组（Variable Length Array，VLA），在我们的实验中为了简化，选择不支持它。请你简要回答，如果我们决定支持一维的可变长度的数组(即允许类似 `int n = 5; int a[n];` 这种，但仍然不允许类似 `int n = ...; int m = ...; int a[n][m];` 这种)，而且要求数组仍然保存在栈上（即不允许用堆上的动态内存申请，如 `malloc` 等来实现它），应该在现有的实现基础上做出那些改动?
> 提示：不能再像现在这样，在进入函数时统一给局部变量分配内存，在离开函数时统一释放内存。你可以认为可变长度的数组的长度不大于0是未定义行为，不需要处理。

Answer：需要的主要改动包括：

1. 引入帧指针：要支持 VLA，则栈指针在运行中是可变的，那么原来基于栈指针的寻址将会在栈指针改变后出错，所以需要用帧指针来寻址参数和 VLA 前声明的局部变量。
2. 动态修改栈指针：在支持 VLA 之前，栈帧在进入函数时统一分配，栈指针只会修改一次，现在需要在遇到 VLA 声明时动态修改栈指针，来为 VLA 分配内存。
3. 修改恢复机制：原来函数运行结束后通过为栈指针增加常数的方式来释放栈帧，现在由于栈指针动态变化，则需要使用帧指针来恢复栈指针，释放函数栈帧。同理，在进入作用域时，需要先记录原来的栈指针，当作用域内包含动态数组声明时，出作用域需要使用原来记录的栈指针来恢复栈指针，从而实现作用域内动态数组的消亡。

Step 12

Question 1

> 作为函数参数的数组类型第一维可以为空。事实上，在 C/C++ 中即使标明了第一维的大小，类型检查依然会当作第一维是空的情况处理。如何理解这一设计?

Answer：C/C++ 中函数参数如果为数组类型，只会传递数组的首指针，而不是拷贝整个数组，从而极大的节省时间和空间；而数组的元素偏移计算只需要第 2～n 维的长度，并不需要第一维

的长度，所以第一维长度在调用中没有实际意义；传参时将第一维长度当作不存在，还能增强函数的灵活性，即同一函数可以支持第一维长度为任意值的数组，提高了代码的可复用性。